

基于 SpringBoot 的企业内购商城系统开发

摘 要

随着互联网的普及和电子商务的发展，在线商城系统已逐渐成为商业领域中不可或缺的重要工具。许多企业对于企业内部的在线商城系统需求日益增长，此情况下设计一个企业内购商城系统不仅能够提升企业内部采购效率，也能够为员工提供便捷的购物体验 and 优惠的购物价格。本系统结合具有易用性与扩展性的 SpringBoot 和 Vue 框架，采用前后端分离的架构，使用 Redis 来进行数据缓存，实现了对普通用户管理、个人信息管理、商品展示、购物车、订单支付、订单处理等系统功能，提供了友好的用户界面和良好的用户体验。该系统在实际应用中还可以提高企业员工的福利，提高员工的归属感，为企业的数字化转型和内部商城建设提供了有力的技术支持。

关键词 企业内购商城；Vue 框架；Redis 缓存；SpringBoot；前后端分离

ONLINE SHOPPING MALL SYSTEM BASED ON THE SPRINGBOOT PROJECT

ABSTRACT

With the popularization of the Internet and the development of e-commerce, the online mall system has gradually become an indispensable tool in the business field. Many enterprises have an increasing demand for internal online shopping mall systems. In this situation, designing an internal shopping mall system can not only improve the efficiency of internal procurement, but also provide employees with a more convenient shopping experience and discounted shopping prices. This system combines the easy-to-use and scalable SpringBoot and Vue frameworks, adopts a front-end and back-end separation architecture, and uses Redis for data caching, achieving system functions such as ordinary user management, personal information management, product display, shopping cart, order payment, and order processing, providing a friendly user interface and a good user experience. In practical applications, this system can also improve the welfare of enterprise employees, enhance their sense of belonging, and provide strong technical support for the digital transformation and internal mall construction of enterprises.

KEY WORDS internal shopping mall for enterprises; Vue framework; Redis cache; SpringBoot; front and rear separation

目 录

摘 要.....	I
ABSTRACT	II
1 绪论.....	1
1.1 研究背景.....	1
1.2 研究现状.....	1
1.3 设计目标与意义.....	1
1.4 论文结构.....	2
2 相关技术介绍	3
2.1 开发语言.....	3
2.2 开发工具.....	3
2.2.1 IDEA	3
2.2.2 Navicat	3
2.2.3 Vs Code	3
2.3 开发技术.....	3
2.3.1 SpringBoot	3
2.3.2 MyBatis	4
2.3.3 MySQL	4
2.3.4 Vue	4
3 系统需求分析.....	5
3.1 系统可行性分析.....	5
3.2 功能性需求分析.....	5
3.2.1 用户用例分析.....	5
3.2.2 账户管理模块.....	6
3.2.3 商品管理模块.....	7
3.2.4 订单管理模块.....	7
3.3 非功能性需求分析.....	7
4 系统设计.....	9
4.1 系统总体架构.....	9
4.2 业务流程设计.....	9
4.3 系统数据库设计.....	11
4.3.1 实体设计.....	11
4.3.2 实体关系.....	14
4.3.3 数据库表设计.....	14

5 系统实现.....	18
5.1 账户管理模块.....	18
5.1.1 注册功能实现.....	18
5.1.2 登录功能实现.....	19
5.1.3 顾客信息管理功能实现.....	20
5.2 商品管理模块.....	21
5.2.1 商城首页功能实现.....	21
5.2.2 商品详情页功能实现.....	22
5.2.3 添加商品功能实现.....	23
5.2.4 购物车功能实现.....	24
5.2.5 商品评价功能实现.....	25
5.3 订单管理模块.....	26
5.3.1 订单功能实现.....	26
5.3.2 地址功能实现.....	27
5.3.3 支付功能实现.....	28
6 系统测试.....	29
6.1 测试目的.....	29
6.2 系统功能测试.....	29
6.2.1 单点登录测试.....	29
6.2.2 商品前台功能测试.....	31
6.2.3 评论功能测试.....	34
6.3 测试结果.....	35
7 系统部署与运维.....	36
7.1 系统部署.....	36
7.2 系统运维.....	36
结 论.....	38
致 谢.....	39
参考文献.....	40

1 绪论

1.1 研究背景

电子商务行业在过去几年取得了巨大的发展，尤其是受到新冠疫情影响，在线购物在全球范围内蓬勃发展，成为了各个企业经营模式中的主流之一[1]。在这样的背景下，许多企业内部员工采购方式也逐渐向线上转移，企业内购商城系统应运而生，开发一个有利于企业内购商城系统成为了一个备受关注的课题。

随着企业的发展和规模的扩大，员工福利和采购管理逐渐成为企业管理的重要方面。传统的员工福利和采购管理方式往往存在着效率低下、成本高昂、管理不便等问题。而企业内购商城系统通过集中采购、批量采购、在线化采购等方式，能够解决这些问题，提高员工福利和采购管理的效率和质量。员工对于购物的需求也在不断变化，他们更加注重购物的便捷性、安全性和个性化服务。企业内购商城系统正是为了满足这些需求而不断发展和完善。它提供了丰富的商品种类和优惠的价格，同时支持在线支付和多种配送方式，让员工能够更加方便、快捷地购买到自己需要的商品。

企业内购商城系统也是企业数字化转型的重要一环。数字化转型已经成为企业发展的必然趋势，而企业内购商城系统作为数字化转型的重要应用之一，能够为企业带来更加高效、便捷、智能的购物体验和管理方式，提高企业的竞争力和市场地位。

1.2 研究现状

企业内购商城系统作为员工福利和采购的重要形式，近年来备受关注。其研究现状显示，随着云计算和大数据技术的普及，企业开始重视商城系统的数字化转型，以提升采购效率和员工满意度。然而，市场竞争激烈，国内外电商平台和本地化平台纷纷提供丰富的商品、优质服务和便捷支付，以吸引企业用户。

技术门槛高是中小型企业面临的一大挑战，商城开发需要掌握前端开发、后端开发、数据库管理等多种技能，这限制了它们的自主开发能力。因此，许多企业依赖外部服务商进行商城的开发和维护。

尽管面临挑战，企业内购商城系统在数字化转型、用户体验、物流与支付方式等方面已取得一定进展。未来，随着技术的不断发展和市场需求的变化，企业内购商城系统将更加高效、便捷和智能，为企业提供更优质的采购解决方案。

1.3 设计目标与意义

随着互联网普及和技术的持续进步，网络购物已深度融入人们的日常生活。特别

是在信息技术不断创新和突破的背景下，网络购物的优势愈发显著，已经逐步取代传统的线下购物方式。

面对物欲横流，物价膨胀，许多商品在第三方平台的价格都很高，企业采购成本又很高，当前许多企业和员工都急需一个属于他们公司的内购商城系统。系统通过数字化手段优化传统采购流程，不仅能大幅提升采购效率，减少成本开支，还能为员工提供更加便捷、个性化的购物体验，同时也能够满足企业内部的实际需求，提升采购管理的现代化水平，也为其他类似企业提供了可借鉴的数字化转型方案。此外，系统还将助力企业实现精准营销和库存管理，推动企业整体运营水平的提升。通过对用户购物行为和偏好的深入分析，系统将实现个性化推荐、智能搜索等功能，为用户提供更加个性化、便捷的购物体验。

基于 SpringBoot 的企业内购商城系统不仅可以满足当前企业和员工对内部商城的需求，还能够为未来企业的经营发展奠定良好的基础^[2]。因此，深入研究并通过 SpringBoot 框架的高效开发特性和微服务架构的灵活性，构建一个稳定、可扩展的企业内购商城系统，以满足现代企业对于内部采购管理的实际需求极为重要，探索其在提升电子商务竞争力和用户体验方面的潜力，具有重要的理论和实践意义。

1.4 论文结构

文章主要介绍企业内购商城系统的设计过程，讲述该系统的需求分析，设计方案，实现过程以及系统测试。

第一章，绪论。介绍课题研究的背景和意义。

第二章，相关技术介绍。介绍项目所用到的开发工具和开发技术。

第三章，系统需求分析。介绍项目的可行性分析和需求分析。

第四章，系统设计。介绍系统的总体设计和数据库设计。

第五章，系统实现。介绍系统主要功能的实现过程。

第六章，系统测试。介绍系统功能的测试和目的。

第七章，系统部署与运维。介绍如何部署项目和项目的运维。

2 相关技术介绍

2.1 开发语言

该系统采用Java 作为基础开发语言。Java 作为一种高级编程语言，已经经历了 20 年的发展，语言完整严谨，在计算机编程和项目开发领域拥有广泛的应用。Java 语言拥有较优的语言特性，面向对象编程是最优秀的特性之一，Java 还具备其他诸多优秀的语言特性。比如，Java 的跨平台性使得程序能够在不同操作系统上运行。这些特性使得 Java 语言成为了一个强大而灵活的工具，广泛应用于各种软件开发项目中，为软件以及各大企业的发展做出了重要的贡献。

2.2 开发工具

2.2.1 IDEA

该系统采用了 IDEA 开发工具。IDEA 是一款极为强大的集成开发环境，由 JetBrains 公司精心塑造而成^[3]。其聚焦于 Java 开发，并且亦对多种其他语言及技术予以支持，从而契合了各类迥异的开发需求。涵盖了智能代码撰写、灵活的调试、便捷的版本控制以及多元的插件支持等等。IDEA 还具有非常强大的调试工具以及性能分析功能，有利于开发者迅速定位并解决问题，增进了代码的质量。

2.2.2 Navicat

该系统采用了 Navicat 数据库工具。Navicat 是一款功能强大的跨平台数据库管理工具。涵盖了广泛的数据库系统，包括 MySQL、PostgreSQL、SQLite、Oracle 和 SQL Server 等，具有直观友好的图形用户界面和丰富的功能集合^[4]。提供了数据库连接、数据查询、编辑功能、数据同步、备份恢复等高级功能^[5]。

2.2.3 Vs Code

该系统采用了 Vs Code 开发工具。全称即 Visual Studio Code，Vs Code 具备极高的可扩展性，带有非常丰富插件和各种有用的功能。Vs Code 的调试功能极为出色，能够协助开发者迅速排查解决代码中的问题。其用户界面简洁、直观，容易上手。它为开发者提供了高效、便捷的开发环境，无论是新手还是经验丰富的开发者都能够从中获益。

2.3 开发技术

2.3.1 SpringBoot

此次项目设计中，选择 SpringBoot 框架作为后端开发的核心框架。SpringBoot

并非通过引入全新的技术来创新，而是将现有的框架技术进行整合，同时为开发者提供了极为方便的全局配置，极大提高了企业内购商城的开发效率^[6]。

通过 **SpringBoot**，开发者可以更加便捷地启动项目，无需过多关注底层的项目配置细节。这种简化的开发模式极大地降低了项目开发的门槛，使得更多的开发者能够轻松地投入到项目的开发中。这种简化开发模式为项目开发提供了更高的效率和便利性，使得项目能够更快地落地和推进。

2.3.2 MyBatis

此次项目设计中，使用的 **MyBatis** 是数据访问层框架，主要支持对 **SQL** 语句的自定义、**SQL** 语句集的管理以及高级的数据库映射。**MyBatis** 的设计初衷是为了帮助开发者有效地管理项目中的 **SQL** 语句，从而实现数据访问与业务逻辑代码的分离^[7]。**MyBatis** 的出现为开发者提供了一种便捷、灵活的数据库访问方式，使得项目的数据库访问逻辑更加清晰、易于管理。通过将 **SQL** 语句与代码分离，**MyBatis** 为开发者提供了一种简洁、高效的数据访问解决方案，极大地提高了项目的开发效率和维护性。

2.3.3 MySQL

此次项目设计中，使用了 **MySQL** 数据库。其作为业界最受欢迎的关系型数据库之一，在社区版中依然是一款广受欢迎的开源数据库管理系统。**MySQL** 以广泛应用的结构化查询语言（**SQL**）来管理数据库，在 **Java** 项目开发中得到了广泛的应用和认可。

MySQL 具有强大的跨语言支持，开发者可以通过调用接口实现多种编程语言的兼容。这种特性使得其在不同的开发环境下都能够轻松地被集成和应用，从而满足不同项目的需求^[8]。**MySQL** 在操作系统的支持上也非常广泛。这也使得 **MySQL** 成为了此次项目中的首选数据库管理系统，为开发者项目设计提供了强大的技术支持。

2.3.4 Vue

此次项目设计中，前端核心架构采用的是 **Vue** 框架。**Vue** 框架是一种“渐进式框架”，专为构建用户界面而设计。相较于其他一些笨重的框架，**Vue** 采用了自底向上逐渐增量的设计理念，使其对于初学者来说更加亲和和易于掌握^[9]。此外，**Vue** 还具备与第三方库或已有项目的无缝整合能力。**Vue** 框架的出现极大地简化了前端开发的流程。使其在此次企业内购商城开发中提供强大的便利和有利价值，发挥了很重要的作用，极大地提升了工作效率和项目的质量^[10]。

3 系统需求分析

3.1 系统可行性分析

(1) 经济可行性

在软件层面上，开发者使用的所有相关开发工具包括开发技术全部都是开源免费，直接在网络上下载安装即可，无需费用支出。在硬件层面上，所有的开发工具皆是在开发者笔记本电脑上完成的，并且开发者电脑配置完全足以满足系统的开发需求。在开发进程中运用了学校所提供的网络，无需再额外去搭建网络环境，在整个开发过程当中并未产生额外的支出费用。综合进行考量，该系统的开发成本是极其低的。综上所述，本系统具有经济可行性。

(2) 技术可行性

该系统的开发是以Java语言为基础的，使用了SpringBoot开发框架。在应用程序的开发进程中。在数据库方面，则采用了MySQL工具来进行搭建，同时与MyBatis持久层框架相结合。这一设计架构让系统的结构变得更加清晰明了，有利于维护以及进行单元测试，而且也成功地实现了SQL与代码的分隔，提升了系统的可维护性以及开发的效率^[11]。Java与SpringBoot的结合为项目的迅速开发、简洁部署和平稳运行提供了强有力的技术保障^[12]。综上所述，本系统具有技术可行性。

(3) 操作可行性

系统的页面设计致力于提供给用户方便、快捷和易用的操作体验，确保用户在使用过程中能够感到舒适。整体页面的设计保持了一致性，包括风格、字体和背景，确保了用户体验的连贯与舒适。同时，在功能按钮上使用的字词应简洁易懂，以避免用户因误操作而产生困惑。在设计过程中，注重细节和用户体验，是确保系统页面设计质量的重要保证。综上所述，本系统具有操作可行性。

3.2 功能性需求分析

3.2.1 用户用例分析

系统会将用户按权限分为管理员和普通用户两种角色，管理员功能用例分以下六个模块：管理员登录、商品管理、订单管理、营销管理、权限管理、个人中心。管理员功能用例图如图3-1所示。

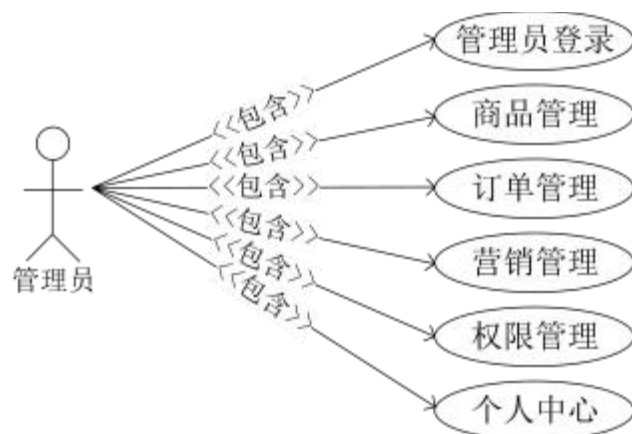


图 3-1 管理员功能用例图

普通用户功能用例分以下七个模块：用户登录、用户注册、购物车、我的订单、个人中心、修改密码、商品评价。用户功能用例图如图3-2 所示。

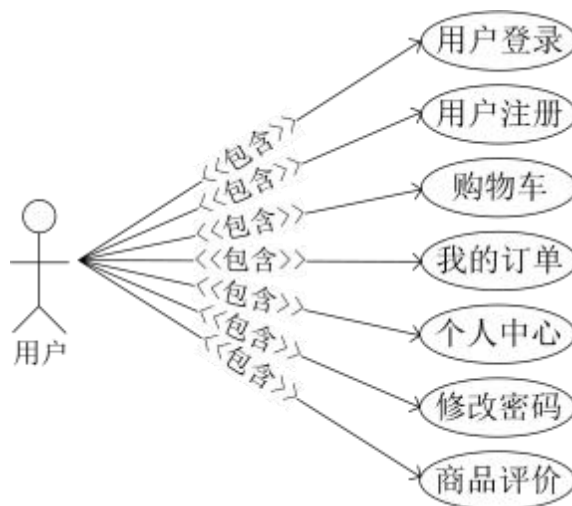


图 3-2 用户功能用例图

3.2.2 账户管理模块

第一，注册登录功能。系统必须具备注册登录功能，普通用户必须使用企业企业邮箱进行注册和登录，而管理员仅具备登录功能。在注册环节中，用户必须确保提交的注册信息，例如用户名、密码和企业邮箱等，都是准确无误的。对于登录功能，用户能够通过其注册信息进若行登录。在登录失败时，系统应提供明确的错误提示，以帮助用户快速定位问题。建立稳固安全的登录系统，从而确保用户的账户安全并优化其使用体验。

第二，个人账号管理功能。普通用户必须具备自身的账号管理功能，账号管理模块的功能性需求分析需要考虑多方面的内容。首先，要能让用户方便地查看并修改账号信息，包括个人资料、安全设置等。要具备密码重置功能，以应对用户忘记密码的情况。普通用户还可在个人中心添加企业内购会员vip，使商城功能更加完善。

第三，顾客信息管理功能。管理员可以管理所有顾客，可以启用或关闭用户的功能，对普通用户进行删除和分配权限。

3.2.3 商品管理模块

第一，商城首页功能。商城首页具有展示所有商品的功能，普通用户可以在商城首页进行搜索、查看、购买商品等操作。

第二，商品详情页功能。系统必须具备商品详情页功能，确保用户能够全面、清晰地了解商品信息，以便做出购买决策。这包括展示商品图片、详细描述、规格参数、用户评价等关键内容，同时提供便捷的购买、加入购物车、分享等交互功能，以满足用户的浏览和购物需求。

第三，商品详情管理功能。管理员具备商品管理功能，系统管理员具有对系统商品实施管理的权力，必须具备商品种类的创建、编辑、删除等操作功能，能够对不同种类的商品进行分类和标识，便于管理和查询；普通用户能够在按照商品分类去查看各式各样的商品信息，购买商品后对商品对商品进行评价等功能。

第四，购物车功能。系统必须具备购物车功能，普通用户仔细地挑选令自己满意的商品，并将其加入购物车。此外，他们还能随心所欲地对购物车里面已有商品展开一系列操作，比如改变数量、删除之类的操作，以满足自身的购物需求。

3.2.4 订单管理模块

第一，订单地址管理功能。普通用户具备地址添加功能，用户购买商品必须选择地址，在商品详情页面应该具备地址添加功能。

第二，订单支付功能。系统必须具备订单支付功能，普通用户提交生成支付订单后，系统需要提供支付接口，可以选用支付宝的沙箱支付功能。

第三，订单详情功能。系统拥有订单详情功能，普通用户可以在订单详情页对商品进行确认收货，评价，退货等操作。管理员可以在后台对商品进行发货操作，退货等操作。以上所有用户操作都离不开订单管理功能。

3.3 非功能性需求分析

第一，系统性能需求。企业内购商城系统需要具备快速响应用户请求的能力，有效的安全措施保护用户数据安全，高可靠性和容错性确保服务持续可用，高效的数据处理能力处理大量商品信息和订单数据^[13]。

第二，安全性需求。企业内购商城系统的安全性需求包括保护用户个人信息，采

取有效的身份认证和访问控制措施防止恶意攻击，加强对系统的漏洞和安全漏洞的监测和防范，确保系统在面对各种网络威胁时保持稳定和可靠。

第三，可维护性需求。企业内购商城系统的可维护性需求包括具备清晰的代码结构和良好的文档说明，方便开发人员进行代码的维护和修改，提供方便的测试和调试工具，以及及时的技术支持和更新，以确保系统能够持续地适应和响应业务需求的变化。

第四，可扩展性需求。企业内购商城系统的可扩展性需求主要体现在能够方便地添加新的功能和模块，以适应业务的增长和变化^[14]。它需要具备灵活的架构，能够容纳更多的商品类别和数量，支持新的支付方式和配送方式等。同时，还应能与其他系统进行有效集成，实现数据的共享和交互^[15]。

4 系统设计

4.1 系统总体架构

企业内购商城系统涵盖了两个关键的功能模块：一是管理员模块，二是用户模块。用户模块主要是面向前台用户，需要用户在进行注册之后才能进行以下操作。该模块整合了多项核心功能，包括商品购买流程、商品详细信息的快速查询、用户个人信息的更新与维护、收货地址的修改与保存、购物车内容的便捷管理，以及订单状态的实时追踪与管理，为用户提供一站式的购物体验。而管理员模块具备后台管理的功能，主要涉及用户以及商品信息的管理。管理员能够对用户信息实施角色分配和删除的操作，还有进行商品信息的增加更改、查询，以及下架删除的各种操作。系统总体业务架构图如图 4-1 所示。

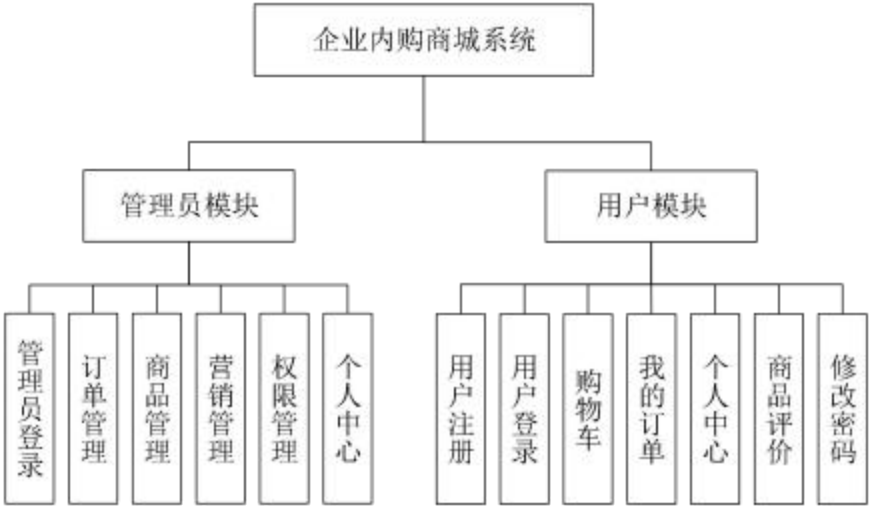


图 4-1 系统业务架构图

4.2 业务流程设计

系统业务流程的设计是借助图文相结合的方式，针对系统的运转流程予以扼要概述。企业内购商城系统里面，主要涵盖了管理员以及用户这两类角色。凭借各不相同的登录界面以及与之相应的角色权限，用户得以执行各类不同的操作。

当用户进入到此系统时，首先必须要进行账号注册，输入相关登录信息，从登录开始系统便会账号权限进行识别，其次才能登陆到系统，用户注册登录功流程图如图 4-2 所示。

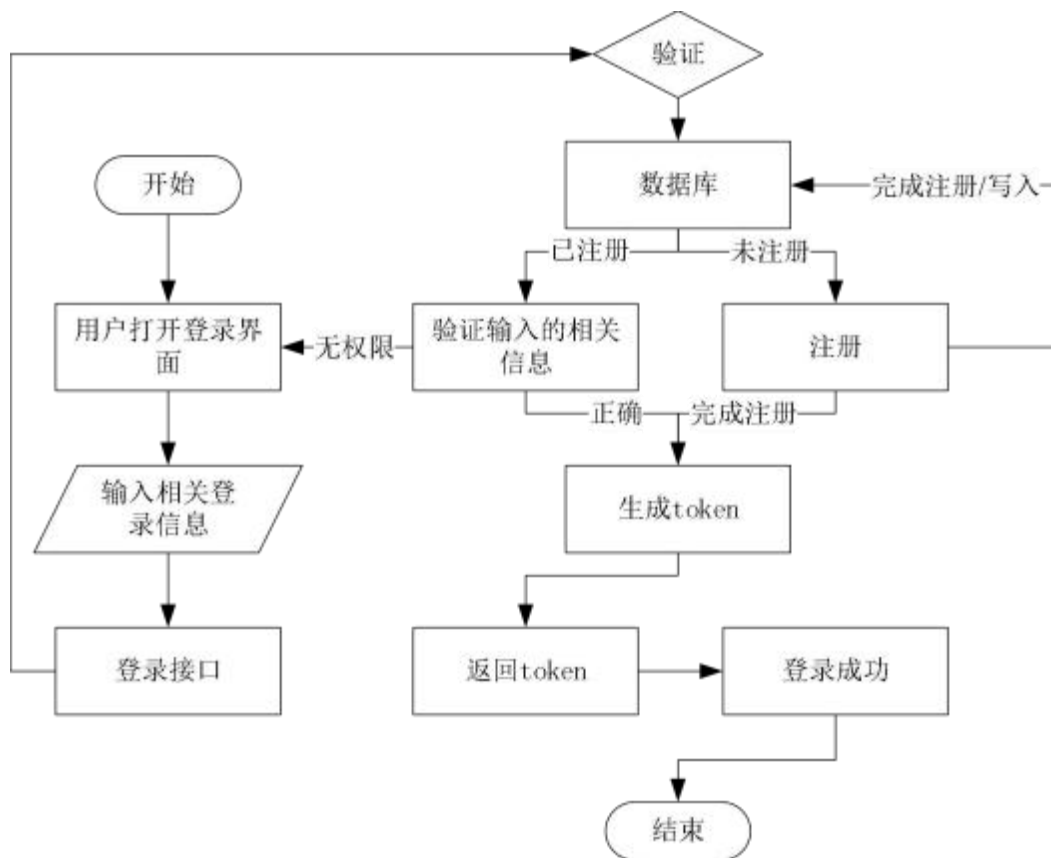


图 4-2 用户注册登录流程图

管理员是企业内购商城系统的管理者，具备着管理商城商品的各种权限，当输入管理员账号的时候，系统才会进入到管理员模块，管理员操作流程如图 4-3 所示。

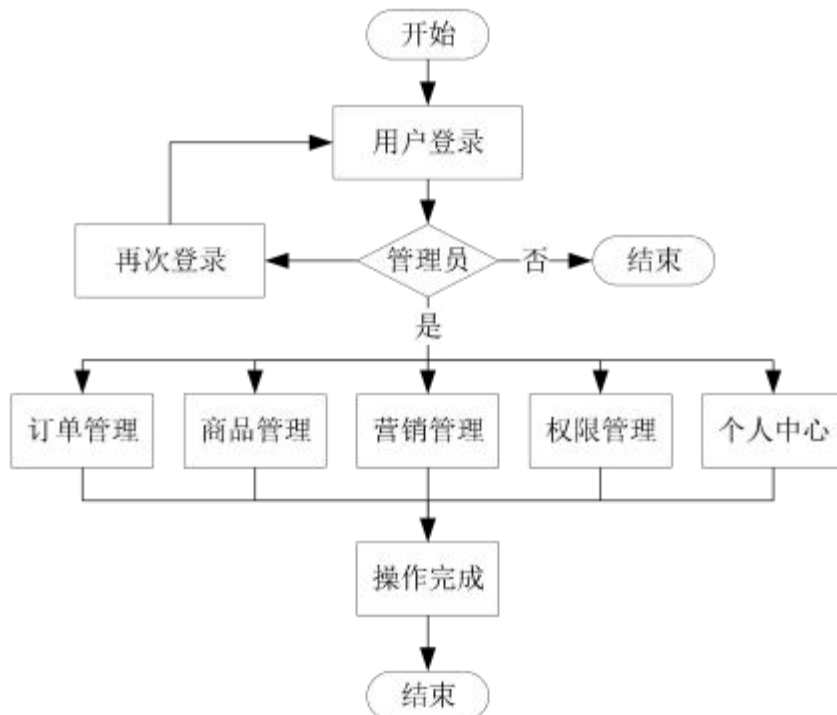


图 4-3 管理员操作流程

用户是企业内购商城的员工，当用户使用企业邮箱登录的时候，系统才会进入到用户模块，然后用户才可以对商品进行操作，普通用户操作流程如图 4-4 所示。

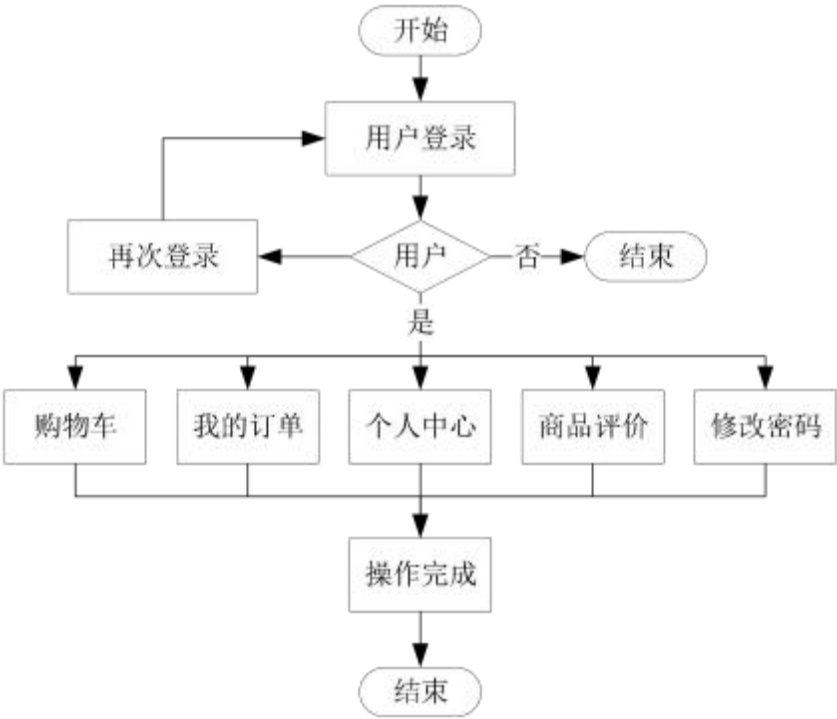


图 4-4 用户操作流程图

4.3 系统数据库设计

4.3.1 实体设计

数据库必须具备角色实体，不同的登录用户拥有不同的编号和权限。角色实体属性图如图 4-5 所示。

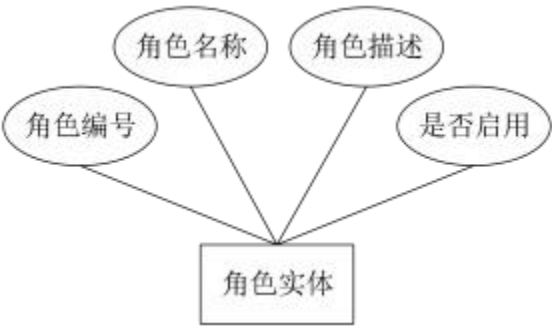
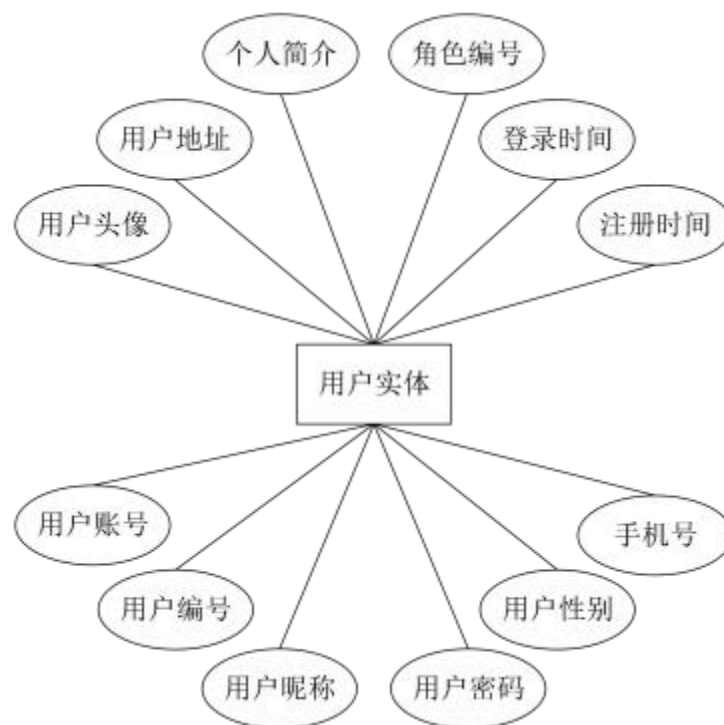


图 4-5 角色实体属性图

数据库必须具备用户实体，包括用户的个人简介、密码、用户账号、手机号等个人信息。用户实体属性图如图 4-6 所示。



数据库必须具备商品信息实体，商品信息需要有与之对应的商品编号、商品名称、和其它商品属性。商品信息实体属性图如图 4-7 所示。



数据库必须具备订单信息实体，用户购买商品后会生成与之对应的订单，每个商品都有相关的订单，因此订单信息实体也不可缺少。订单信息实体属性图如图 4-8 所示。

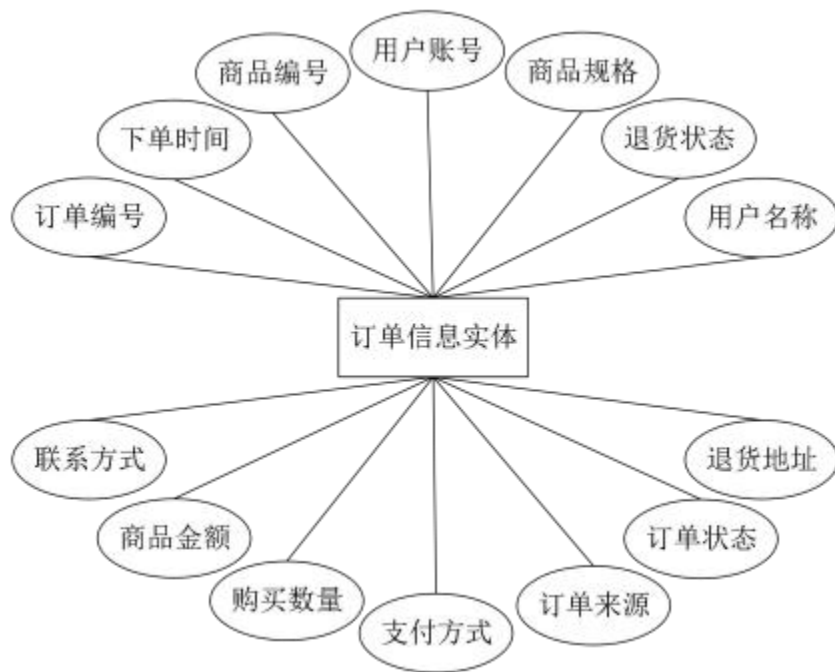


图 4-8 订单信息实体属性图

数据库必须具备商品类别实体，商城所有商品可以通过类别进行分类搜索，由此必须设计商品类别实体。商品类别实体属性图如图 4-9 所示。

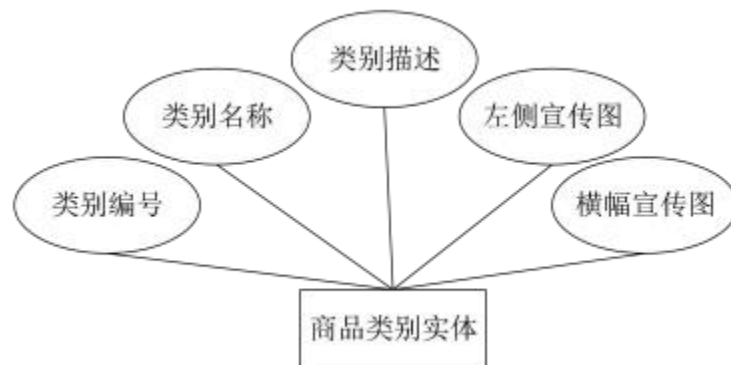


图 4-9 商品类别实体属性图

数据库必须具备商品品牌实体，可以对每个上架的商品进行品牌分类，商品品牌实体属性图如图 4-10 所示。



图 4-10 商品品牌实体属性图

数据库必须具备商品评价实体，商品确认收货后，可以对相应的订单进行评价。商品评价实体属性图如图 4-11 所示。



图 4-11 商品评价实体属性图

4.3.2 实体关系

数据库 E-R 图设计是以系统用户与业务之间所存在的实体联系作为基础来进行展开分析的，是系统数据库概念模型，通过 E-R 图可以清晰明确地确定数据库实体相互之间的关系。这些实体联系的范围包含了一对一、多对一、一对多以及多对多等各种形式。如一个用户可以购买多个商品、发表多条商品评价、查看多条订单信息，一个管理员可以管理商品信息、订单信息、商品类别，一个管理员可以修改商品品牌、并且所有用户和管理员都是一个角色。数据库 E-R 图如图 4-12 所示。



图 4-12 数据库 E-R 图

4.3.3 数据库表设计

(1) 角色表(role): 根据系统针对管理员模块的需求来看。表应当包含：角色编号、角色名称、角色描述、是否启用。角色表具体字段如表 4-1 所示。

表 4-1 角色表(role)

字段	类型	长度	描述
role_id	int	20	角色编号
role_name	varchar	30	角色名称
role_describe	varchar	50	角色描述
role_state	tinyint	1	是否启用

(2) 用户表(user): 根据系统对用户表的设计需求来看, 用户表应包含用户编号、注册时间、性别、账号、昵称等。用户表具体字段如表 4-2 所示。

表 4-2 用户表(user)

字段	类型	长度	描述
user_id	int	20	用户编号
account_number	varchar	30	用户账号
user_name	varchar	30	用户昵称
password	varchar	50	用户密码
user_sex	varchar	6	用户性别
telephone	varchar	11	用户手机号
creat_time	timestamp	20	注册时间
login_time	timestamp	20	登录时间
role_id	int	20	角色编号
summary	varchar	50	个人简介
user_address	varchar	100	用户地址
avatar_url	varchar	200	用户头像

(3) 商品信息表(product): 此系统数据库必须具备商品信息表, 商品信息表应当含有: 商品编号、商品名称、商品类别等。商品信息表具体字段如表 4-3 所示。

表 4-3 商品信息表(product)

字段	类型	长度	描述
product_no	varchar	30	商品编号
product_name	varchar	30	商品名称
product_type	varchar	30	商品类别
product_describe	varchar	100	副标题
product_brand	varchar	30	品牌
in_price	double	10	进价

续表

字段	类型	长度	描述
out_price	double	10	售价
product_stock	int	20	库存
lowest_stock	int	50	最低库存
is_stockout	tinyint	1	是否缺货
is_new	tinyint	1	是否新品
is_sale	tinyint	1	是否上架
sale_time	timestamp	20	上架时间
product_url	varchar	200	商品图片

（4）商品类别表(product_type): 根据实体设计中的商品类别来看，此系统数据库必须具备商品类别表，商品类别表应当含有：类别编号、类别名称、类别描述、左侧宣传图、横幅宣传图。商品类别表具体字段如表 4-4 所示。

表 4-4 商品类别表(product_type)

字段	类型	长度	描述
type_id	int	10	类别编号
type_name	varchar	30	类别名称
type_describe	varchar	100	类别描述
type_url_left	varchar	200	左侧宣传图
type_url_top	varchar	200	横幅宣传图

（5）商品品牌表(product_brand): 此系统数据库必须具备商品品牌表，商品品牌表应当含有：品牌编号、品牌名称、品牌描述。商品品牌表具体字段如表 4-5 所示。

表 4-5 商品品牌表(product_brand)

字段	类型	长度	描述
brand_id	int	10	品牌编号
brand_name	varchar	30	品牌名称
brand_describe	varchar	100	品牌描述

（6）商品订单表(order): 此系统数据库必须具备商品订单表，商品订单表应当包含：订单编号、下单时间、商品编号、商品规格等。商品订单表具体字段如表 4-6 所示。

表 4-6 商品订单表(order)

字段	类型	长度	描述
order_no	varchar	30	订单编号
order_time	timestamp	30	下单时间
product_no	varchar	30	商品编号
product_specs	varchar	30	商品规格
user_account	varchar	30	用户账号
user_name	varchar	30	用户名称
contact_way	varchar	30	联系方式
pay_price	double	20	商品金额
pay_amount	int	10	购买数量
pay_type	varchar	20	支付方式
order_from	varchar	20	订单来源
order_state	varchar	20	订单状态
accept_address	varchar	100	收货地址
return_state	tinyint	1	退货状态

(7) 商品评价表(product_review): 由于商品评论留言对用户具有很重要的参考价值, 本系统商品评价表应当涵括: 用户评论编号、用户帐号、商品编号、商品规格、订单编号、评论时间、商品评星、商品评价。商品评价表具体字段如表 4-7 所示。

表 4-7 商品评价表(product_review)

字段	类型	长度	描述
review_id	int	10	评论编号
account_number	varchar	30	用户帐号
product_no	varchar	30	商品编号
product_specs	varchar	30	商品规格
order_no	varchar	30	订单编号
review_time	timestamp	10	评论时间
star_level	double	10	商品评星
product_review	varchar	300	商品评价

5 系统实现

5.1 账户管理模块

5.1.1 注册功能实现

系统用户注册时必须使用公司OA系统的企业邮箱才能进行注册，用户注册账号时依次输入昵称、企业邮箱、密码，然后点击发送验证码，随后企业邮箱里便会收到注册账号的验证码，输入验证码后点击注册按钮即可。用户注册界面如图 5-1 所示。

The image shows a web interface for a company's internal purchase system. On the left, there's a header '企业内购商城' and three service highlights: '自由' (Free) with a shopping cart icon, '方便' (Convenient) with a speech bubble icon, and '高效' (Efficient) with a rocket icon. On the right, there's a '注册' (Register) section. It includes a link '已有账号? 点此登录' (Already have an account? Click here to login). The registration form has four input fields: '请输入昵称' (Please enter nickname), '请输入邮箱' (Please enter email), '请输入密码' (Please enter password), and a '邮箱验证码' (Email verification code) field. There's a '发送验证码' (Send verification code) button next to the verification code field. At the bottom of the form is a large blue '注册' (Register) button.

图 5-1 用户注册

系统注册功能主要通过调用 `mailService.sendTemplateMail()` 方法，使用指定的模板发送邮件通知用户注册成功。如果发送成功，函数返回一个包含成功信息的通用结果对象。如果发送失败，函数返回一个包含错误信息的通用结果对象。注册功能实现核心代码如下：

```
public CommonResult sendRegisterInfo(String email, String name, String password)
{
    try {
        Map<String, Object> data = new HashMap<>(4);
        data.put("account", email);
        data.put("name", name);
        data.put("password", password);
        data.put("year", Calendar.getInstance().get(Calendar.YEAR));
        mailService.sendTemplateMail(email, "注册成功", "register", data);
        return CommonResult.success("注册信息发送成功");
    }
}
```

```

    } catch (Exception e) {
        return CommonResult.error("注册信息发送失败");}
    }
}

```

5.1.2 登录功能实现

用户登录系统的时候，根据系统提示需要输入 OA 系统中的企业邮箱账号和登录密码，然后点击登录按钮。如果用户忘记密码也可以点击界面右下角找回密码，然后根据系统提示使用企业邮箱找回。用户登录界面如图5-2 所示。



图 5-2 用户登录

在用户登录过程中，与 el-form 表单组件相关联的表单数据，会被自动传递到 submit Form 方法之中，从而实施严格的格式校验操作。紧接着，这部分数据将会通过 operate Controller.java 文件，进入到用户登录的业务逻辑处理环节。完成这一步骤之后，数据会被迅速传送至数据库的 user 表中。通过这样的方式，可以有效地识别和排除错误或不准确的数据。最终，系统能够判断用户是否能够成功登录进去。这一系列的操作流程，构成了一个完整的用户登录验证机制。登录功能核心代码如下：

```

List<Role> roles = userService.getRoleList(user.getUserId());
Set<String>roseNames=
    roles.stream().map(Role::getRoleName).collect(Collectors.toSet());
Set<String>roseDescribes=
    roles.stream().map(Role::getRoleDescribe).collect(Collectors.toSet());
info.put("role", roseNames);
info.put("roleInfo", roseDescribes);
return CommonResult.success("登录成功", info);

```

5.1.3 顾客信息管理功能实现

后台顾客信息管理界面主要展示普通用户账号、姓名、手机号等关键信息，管理员可以对普通用户进行搜索查询、删除、分配角色操作，也能够选择是否启普通用户权限。后台顾客信息管理如图 5-3 所示。



图 5-3 后台顾客信息管理

后台顾客信息管理中的顾客信息删除功能是通过 UserController.java 类来接收前台传来的重要参数，删除用户的函数内部调用了 userService.deleteById 方法来尝试删除用户，如果删除成功，则返回一个包含成功信息和删除的用户编号的 CommonResult 对象；如果删除失败，则返回一个包含错误信息的 CommonResult 对象。顾客信息删除功能核心代码如下：

```
@RequestMapping(value = "/user/delete")
public CommonResult delete(Integer userId) {
    if (userService.deleteById(userId)) {
        return CommonResult.success("删除成功", userId);
    }
    return CommonResult.error("删除失败");
}
```


5.2 商品管理模块

5.2.1 商城首页功能实现

企业内购商城首页，能够迅速的浏览所有商品，简洁明了的导航栏和商品查询可以轻松找到所需商品。商城首页最上部分是商品搜索框，右上角是个人中心，下部分是商品的营销展示。商城首页如图5-4 所示。



图 5-4 商城首页

商城首页查询功能函数通过调用 `productType Service.selectAllName()` 方法来查询商品类型的名称，并将结果封装在一个 `Common Result` 对象中返回。如果查询成功，该对象会包含一个包含所有商品类型名称的列表以及成功消息；如果查询失败，会返回一个错误消息。该方法使用 `@RequestMapping` 注解来映射 URL，使得可以通过该 URL 来访问该方法。商城首页查询功能核心代码如下：

```
@RequestMapping(value = "/productType/findAllName")
public CommonResult findAllTypeName() {
    List<String> names = productTypeService.selectAllName();
    if (names != null) {
        return CommonResult.success("商品分类名称查询成功", names);
    }
    return CommonResult.error("商品分类名称查询失败");
}
```

5.2.2 商品详情页功能实现

商品详情页是展示商品详细信息的重要页面，此界面主要展示商品详情图、商品名称、商品文字描述、当前库存、企业内购价格、第三方平台价格、收货地址、商品参数、加入购物车和立即购买按钮等关键信息。页面右上角还可以查看其它用户对商品的评价。商品详情页如图5-5 所示。



图 5-5 商品详情页

商品详情页的模块是通过 `MallPurchase.vue` 文件来进行具体实现的。下面这段 Vue 模板代码展示了商品的价格信息，包括当前库存和企业内购价格以及第三方平台价格。其中使用了 Vue 的条件渲染指令 `v-if` 和 `v-else` 来根据商品库存的不同显示不同的库存信息。使用了 Element UI 的分隔符组件 `el-divider` 来分隔不同的价格信息。商品详情页核心代码如下：

```
<div class="price-info">
  <span v-if="productInfo.productStock===0">暂时无货</span>
  <span v-else>当前库存 {{productInfo.productStock}}</span>
  <el-divider direction="vertical"></el-divider>
  <span>企业内购价格： {{productInfo.outPrice}} 元</span>
  <el-divider direction="vertical"></el-divider>
  <span>第三方平台价格： {{productInfo.inPrice}} 元</span>
</div>
```

5.2.3 添加商品功能实现

添加商品界面中，系统管理员在添加商品的时候需要填写商品名称、分类、品牌、三方价、内购价、规格、商品介绍和图片等关键信息，然后点击添加商品按钮即可。后台添加商品界面如图5-6 所示。



图 5-6 后台添加商品

添加商品功能模块依托 `ProductController.java` 类接收前台传递的信息数据，借助 `CommonResult add` 方法完成商品添加的业务逻辑处理，通过 `@RequestMapping` 注解指定请求路径为 `/product/add`。函数接收一个 `Product` 对象作为参数，将其添加到数据库中。如果添加成功，返回一个包含成功消息和添加的商品对象的 `CommonResult`；如果添加失败，返回一个包含错误消息的 `CommonResult`。后台添加商品功能核心代码如下：

```
@RequestMapping(value = "/product/add")
public CommonResult add(Product product) {
    if (productService.insertData(product)) {
        return CommonResult.success("添加商品成功", product);
    }
    return CommonResult.error("添加商品失败");
}
```

5.2.4 购物车功能实现

系统购物车界面主要展示加入商品的价格、规格、数量、单价等重要信息，普通用户可以添加商品到购物车中然后点击结算按钮合计结账，也可以通过点击购物车界面中的垃圾箱按钮删除单独商品或是全部商品，购物车界面如图5-7 所示。

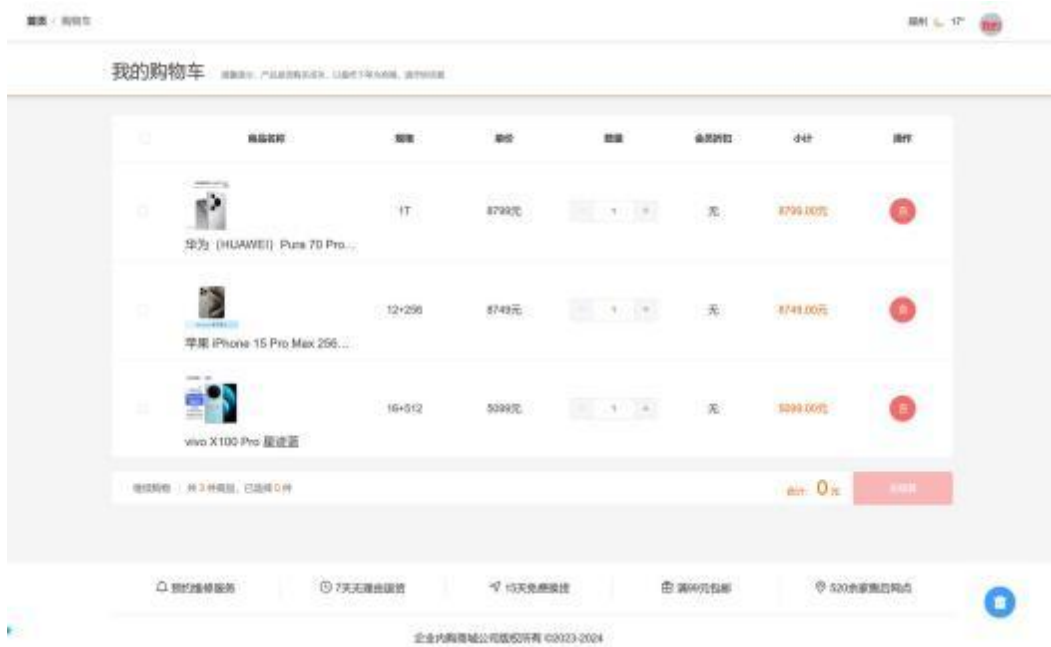


图 5-7 我的购物车

购物车删除功能是通过 `ShoppingCartController.java` 类来响应前台传来的信息，接收购物车商品编号(`cartId`)作为参数。调用 `shoppingCartService.deleteById(cartId)`方法尝试删除指定编号的商品。如果删除成功，函数返回一个成功响应，包含删除成功的消息和删除的商品编号。如果删除失败，函数返回一个错误响应，包含删除失败的消息。购物车删除功能核心代码如下：

```
@RequestMapping(value = "/shoppingCart/deleteById")
```

```
public CommonResult deleteShoppingCart(Integer cartId) {  
    if (shoppingCartService.deleteById(cartId)) {  
        return CommonResult.success("购物车删除成功", "cartId: " + cartId);  
    }  
    return CommonResult.error("购物车删除失败");  
}
```

5.2.5 商品评价功能实现

普通用户在购买商品后可以对商品进行主观评价，商品评价时需要填写具体的评价内容，选择评价星级，然后点击确定按钮即可。商品评价如图5-8 所示。

图 5-8 商品评价

商品评价功能通过`@RequestMapping`注解，指定了请求的 URL 为`/productReview/add`。接收一个 `ProductReview` 对象作为参数，调用 `productReviewService.insertData()` 方法将评论信息保存到数据库。如果保存成功，更新相关订单的状态为评价状态。如果保存失败，则返回一个表示错误的响应。商品评价功能核心代码如下：

```
@RequestMapping(value = "/productReview/add")
```

```
public CommonResult addProductReview(ProductReview productReview) {  
    if (productReviewService.insertData(productReview)) {  
        Integer orderId = orderService.selectIdByKey(productReview.getOrderNo());  
        Order order = new Order();  
        order.setOrderId(orderId);  
        order.setOrderState(EVALUATION_STATE);  
        orderService.updateById(order);  
        return CommonResult.success("商品评论添加成功", productReview);  
    }  
    return CommonResult.error("商品评论添加失败");  
}
```

5.3 订单管理模块

5.3.1 订单功能实现

普通用户可以在订单页面查看全部商品信息，页面提供商品查询功能，有待付款、待发货、待收货订单分类。用户也可以在订单界面执行退货、评价、退款联系客服等操作。订单详情如图 5-9 所示。



图 5-9 订单详情

订单模块中的订单查询功能前台信息数据由 `OrderController.java` 类接收，并进行有关业务逻辑处置，通过 `@RequestMapping` 注解指定请求路径为 `/order/findAll`。函数调用 `orderService.selectAll()` 方法查询所有订单信息，如果查询结果不为空，则返回一个包含查询结果和成功消息的 `CommonResult` 对象；否则返回一个包含错误消息的 `CommonResult` 对象。订单信息核心代码如下：

```
@RequestMapping(value = "/order/findAll")

public CommonResult findAllOrder() {
    List<Order> orders = orderService.selectAll();
    if (orders != null) {
        return CommonResult.success("订单信息查询成功", orders);
    }
    return CommonResult.error("订单信息查询失败");
}
```

5.3.2 地址功能实现

用户购买商品时需要填写收货地址，地址界面第一栏为选择所在地区，第二栏为填写详细地址。填写收货地址如图5-10 所示。

A screenshot of a web form titled "收货地址" (Shipping Address) with a close button (X) in the top right. The form contains two main input fields: "所在地区" (Location) and "详细地址" (Detailed Address). The "所在地区" field is a cascader component showing a selected path: "河南省 郑州市 新郑市" (Henan Province, Zhengzhou City, Xinzheng City). The "详细地址" field is an input component with the text "郑州西亚斯学院" (Zhengzhou Siyasia College) and an edit icon (pencil) on the right. At the bottom right, there are two buttons: "取消" (Cancel) and "确定" (Confirm).

图 5-10 填写收货地址

系统地址功能代码定义了一个 el-dialog 对话框组件来显示地址信息。包含一个表单(el-form)，用于输入和验证地址信息。表单中有两个字段：所在地区和详细地址。所在地区使用 el-cascader 组件选择，详细地址使用 el-input 组件输入。地址功能核心代码如下：

```
<el-form :model="addressForm" :rules="addressRules" ref="addressForm">
  <el-form-item label="所在地区" prop="selectedOptions">
    <el-cascader size="large" :options="options" separator=" "
      v-model="addressForm.selectedOptions">
    </el-cascader>
  </el-form-item>
  <el-form-item label="详细地址" prop="detailAddress">
    <el-input v-model="addressForm.detailAddress" autocomplete="off" style="width:
auto">
      <i class="el-icon-edit el-input__icon" slot="suffix"/>
    </el-input>
  </el-form-item>
</el-form>
```

5.3.3 支付功能实现

普通用户购买商品时可以通过支付宝进行支付，该系统的支付测试功能使用的是支付宝的沙箱支付，支付界面如图 5-11 所示。



图 5-11 支付界面

系统支付功能使用 `@PostMapping` 注解指定请求方法为 POST，`value="/notify"` 指定请求路径为 `/notify`。方法的参数是一个 `Map<String, String>` 类型的参数 `map`，用于接收支付通知参数。支付核心代码如下：

```
@PostMapping(value = "/notify")
```

```
public void payNotify(@RequestParam Map<String, String> map) {  
    if (TRADE_SUCCESS.equals(map.get(TRADE_STATUS))) {  
        String payTime = map.get(TRADE_TIME);  
        String tradeNo = map.get(OUT_TRADE_NO);  
        String tradeName = map.get(TRADE_NAME);  
        String payAmount = map.get(TRADE_AMOUNT);  
  
        log.info("[支付成功] {交易时间: {},订单号: {},订单名称: {},交易金额: {}",  
payTime, tradeNo, tradeName, payAmount);  
    }  
}
```


6 系统测试

6.1 测试目的

本项目完成后需要进行相关测试，测试的关键目的是证明一个程序存在问题，而绝非证明程序是毫无瑕疵的。本章节主要进行系统功能测试来验证系统主要功能是否存在问题。

6.2 系统功能测试

6.2.1 单点登录测试

用户或者管理员输入登陆账号和密码登录的时候，在对相关功能进行相应的操作时必须测试其是否满足基本业务流程。登录功能详细测试用例信息如表 6-1 所示。

表 6-1 登录模块测试用例

测试编号	测试名称	测试描述	预期结果	实际结果
1	用户名密码空值测试	登录时用户名与密码输入值为空	无法进入并提示信息有误	与预期结果相同
2	用户名空值测试	登录时用户名输入为空值	无法进入并提示信息有误	与预期结果相同
3	密码值为空测试	登录时密码输入为空	无法进入并提示信息有误	与预期结果相同
4	错误输入测试	登录时输入错误的用户名和密码	无法进入并提示信息有误	与预期结果相同
5	简单sql注入测试	登录时根据登录时可能的用到的数据库字段进行简单的SQL攻击	无法进入并提示信息有误	与预期结果相同
6	使用用户登录测试	使用正确用户进行登录系统	可以进入并显示相应的系统列表	与预期结果相同
7	使用管理员登录测试	使用正确管理员登录本系统	可以进入并显示相应的界面	与预期结果相同

用户登录模块，当正确输入用户账号和密码，然后点击登录按钮的时候，系统无错误提示并且能够正常登录系统。测试结果如图 6-1 所示。



图 6-1 用户登录测试

用户登录时，当输入错误账号或者密码，或者账号密码为空时，点击登录按钮，系统提示用户名或密码错误，无法登录系统，与测试预期结果相同。测试结果如图 6-2 所示。



图 6-2 用户登录测试

管理员登录的时候，当输入正确的管理员账号和密码，然后点击下方登录按钮的时候，系统无错误提示并且能够正常登录系统。测试结果如图 6-3 所示。



图 6-3 管理员登录测试

6.2.2 商品前台功能测试

系统普通用户登陆后在商品前台对相关功能进行相应的操作时必须要测试其是否满足基本业务流程。商品前台功能详细测试用例信息如表 6-2 所示。

表 6-2 商品前台业务操作测试用例

测试编号	测试名称	测试描述	预期结果	实际结果
1	首页数据加载测试	测试首页数据是否成功加载	加载成功	与预期结果相同
2	商品搜索测试	测试是否可以进行搜索商品	搜索成功	与预期结果相同
3	购物车加入测试	测试商品能否加入购物车	加入成功	与预期结果相同
4	购物车移除测试	测试商品能否移除购物车	移除成功	与预期结果相同
5	个人信息修改测试	测试能否修改个人信息	修改成功	与预期结果相同
6	开通会员测试	测试是否能够开通商城会员	开通成功	与预期结果相同

用户登录成功后，能够清晰地查看到首页数据已经成功加载，与测试预期结果相同。测试截图如图 6-4 所示。



图 6-4 首页加载

用户登录成功后，首页数据已经成功加载，然后进行商品搜索，在搜索栏输入精确数据之后查看到搜索结果与输入商品名称对应，与测试预期结果相同。测试结果如图 6-5 所示。



图 6-5 搜索结果

购物车添加和移除与测试预期结果相同。测试结果如图6-6 所示。

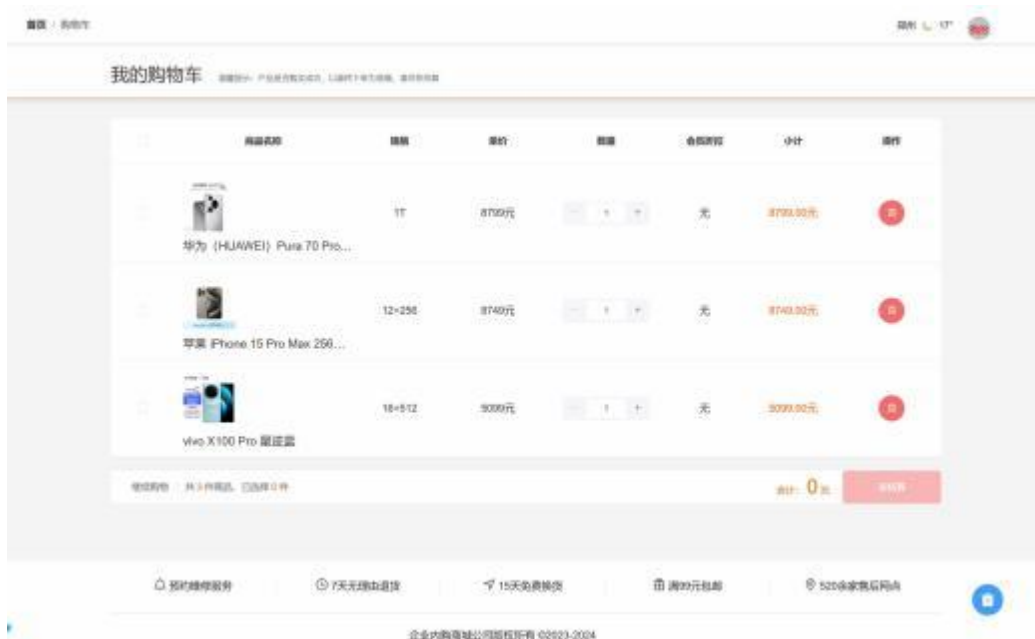


图 6-6 商品添加和移除

修改个人信息成功与测试预期结果相同。测试结果如图6-7 所示。

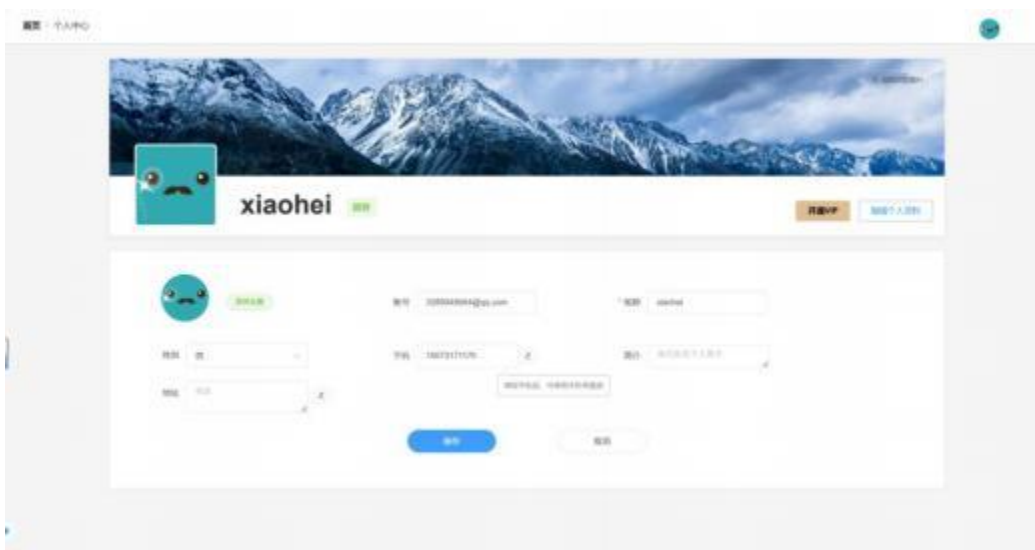


图 6-7 个人信息修改

用户可以开通商城会员，点击开通企业内购 vip，开通成功，与测试预期结果相同。测试结果如图 6-8 所示。

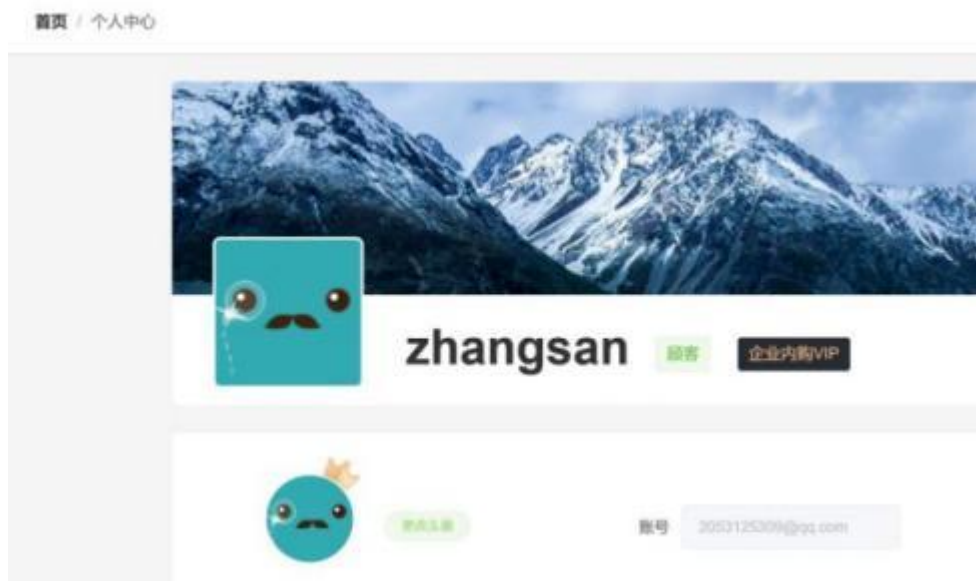


图 6-8 开通会员

6.2.3 评论功能测试

用户购买成功后需要对用户的评论和收货功能进行测试。评论功能详细测试用例信息如表 6-3 所示。

表 6-3 收货评论测试用例

测试编号	测试名称	测试描述	预期结果	实际结果
1	用户收货测试	测试用户是否能够进行收货	收货成功	与预期结果相同
2	用户评论测试	测试用户是否能够进行评论	评价成功	与预期结果相同
3	评价信息展示测试	测试用户评论是否能够成功展示	展示成功	与预期结果相同

点击确认收货按钮，可以查看到系统有收货成功的提示，与测试预期结果相同。测试结果如图6-9 所示。



图 6-9 收货成功

完成的订单后面有评价按钮，点击评价后系统会显示商品评价提交成功，与测试预期结果相同。测试结果如图6-10 所示。



图 6-10 评价成功

在订单界面选择对应的商品，然后点击商品评价，可以清楚的查看到评价的内容。与测试预期结果相同。测试结果如图 6-11 所示。

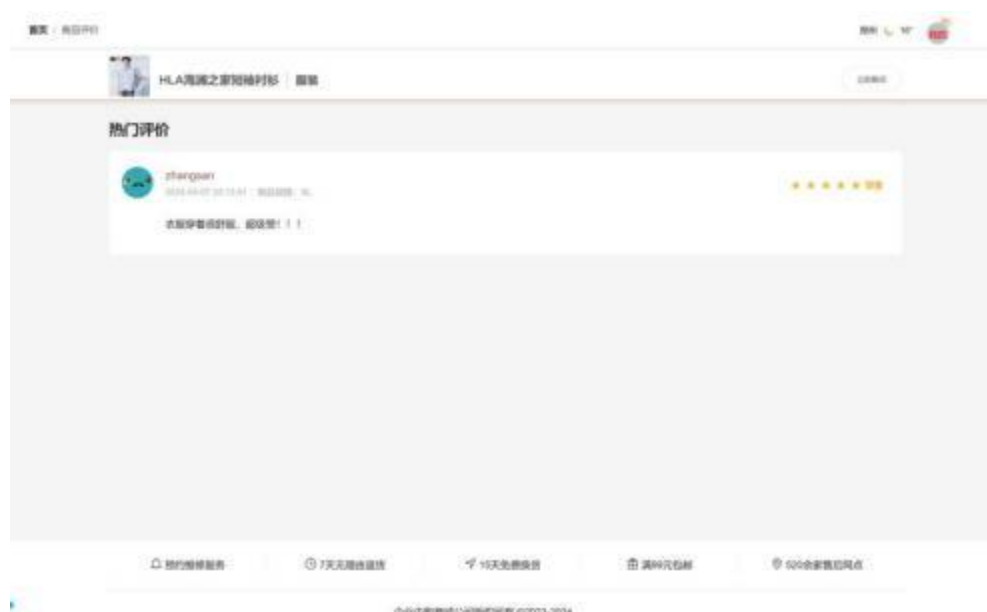


图 6-11 评价展示

6.3 测试结果

在本次商城系统的测试结果中，系统各项功能均表现良好。系统的功能完整性、稳定性以及安全性均成功通过了验证，基本实现了项目初始的设计目的。

7 系统部署与运维

7.1 系统部署

(1) 前端(Frontend)

前端部分是用户与企业内购商城系统直接交互的界面，由浏览器直接展示。

技术栈：Vue.js

部署位置：部署在 Web 服务器上。

(2) 后端(Backend)

后端处理业务逻辑，负责数据处理、用户认证、订单管理等核心功能。

技术栈：SpringBoot 框架、Java 语言、MyBatis。

部署位置：部署在应用服务器上。

(3) 数据库(Database)

数据库用于存储商品信息、用户数据、订单信息等数据。

技术栈：MySQL

部署位置：部署在数据库服务器上，以确保数据安全和性能。

(4) 缓存(Cache)

缓存用于提升系统性能，减轻数据库负载，常用于存储热门商品信息、用户登录状态等。

技术栈：Redis

部署位置：部署在单独的 Redis 缓存服务器上，与数据库服务器分开部署。

(5) 消息队列(Message Queue)

消息队列用于异步处理订单、通知等耗时操作，提高系统吞吐量和可扩展性。部署在独立的消息队列服务器上，与应用服务器分离。

(6) 安全控制(Security Control)

安全控制模块负责用户认证、授权、安全漏洞防护等功能，保护系统免受攻击。

部署位置：集成在后端应用服务器中。

(7) 监控与日志(Monitoring & Logging)

监控与日志模块用于实时监控系统运行状态、收集日志信息，帮助发现和解决问题。部署在独立的监控服务器上，收集和展示系统各项指标数据。

7.2 系统运维

系统运维在项目中有着举足轻重的地位，其牵涉到系统的稳定性、安全性以及可扩展性等诸多方面。所以必须构建一组完备的监控系统，涵盖对系统运行状态、性能指标的实时监控以及针对异常状况的报警机制。还需按时展开系统优化工作，例如性能调优、代码优化等，提升系统的性能与用户体验。通过不断优化系统架构和运维流程，确保企业内购商城系统能够持续稳定、安全地运行，为用户提供优质的购物体验。

结 论

本文详细阐述了企业内购商城系统的设计与实现过程。系统采用 **SpringBoot** 和 **B/S** 架构模式进行功能的开发与设计，只需要有一个支持标准 **HTML** 的浏览器，就可以访问该系统，无需安装任何专门的客户端软件，极大的方便了企业内部的物资采购。

在企业内购商城系统的设计流程中，从选题的时候就广泛借鉴了市面上各类网购商城的运作模式和实践经验，并且对网络上众多开源平台进行了深入的研究。这一过程中，并非简单复制或模仿，而是通过对这些资源和经验的整合与提炼，为企业的内购商城系统注入了独特的思考和设计元素。而且使用 **Vue** 前后端分离的开发技术具有很大的优势，可以提供简单快速的前端开发体验，这进一步加快了企业内购商城系统的开发设计速度。

随着 **Java** 在国内外 **Web** 开发领域的广泛应用，可以选择的开发工具日益丰富。在本次项目中，主要选用了高效的开发工具 **IntelliJ IDEA**，相较于之前常用的 **Eclipse**，它在编码效率上有了显著提升，大大缩短了项目的开发周期，这不仅提高了项目的工作效率，也为系统的稳定性和可靠性提供了有力保障。该系统不仅能够提升企业内部采购效率，也能够为员工带来更多福利，提供更为便捷的购物体验，随着技术的不断发展和市场需求的变化，企业内购商城系统将更加高效、便捷和智能，为企业提供更优质的采购解决方案。

致 谢

在完成这篇论文之际，我满怀感激之情，要向所有在我学术道路上给予我帮助、支持和鼓励的人致以最深的谢意。

我要衷心感谢我的导师郑少彬。他的悉心指导和严格要求，使我在学术道路上不断前进。他的博学、严谨和耐心，让我深受启发，为我的研究提供了无尽的灵感和坚实的基础。感谢我的同学和朋友们，在论文写作过程中，我们相互鼓励、相互支持，共同度过了许多难忘的时光。他们的陪伴和帮助，让我在困难面前始终保持积极的态度。他们为我提供了丰富的文献资源和良好的学习环境。感谢我的母校和图书馆的工作人员，他们为我提供了丰富的文献资源和良好的学习环境。最后感谢我的家人，他们一直以来对我的支持和理解，是我前进的动力和坚实的后盾。

最后，我要感谢所有参与论文评审和答辩的专家和老师们。您的宝贵意见和建议让我的论文更加完善。我会认真听取您的建议，不断进步、追求卓越。

学术之路漫漫，我将铭记这份感恩之情，继续努力，以回报大家的关爱和期望。

参考文献

- [1] 王建, 罗政, 张希, 等.Web 项目前后端分离的设计与实现[J].软件工程, 2020, 23(04):22-24.
- [2] 熊永平.基于 SpringBoot 框架应用开发技术的分析与研究[J].电脑知识与技术, 2019, 15(36):76-77.
- [3] 刘亚茹, 张军.Vue.js 框架在网站前端开发中的研究[J].电脑编程技巧与维护, 2022, (01):18-19+39.
- [4] 喻佳, 吴丹新.基于 SpringBoot 的 Web 快速开发框架[J].电脑编程技巧与维护, 2021, (09):31-33.
- [5] Martin Heller.Redis 6: A high-speed database,cache,and message broker[J].Info World.com, 2020.
- [6] 贺紫珺.基于 SpringBoot和 Vue 框架的第三方医疗器械供应链平台的设计与实现[D].东华大学, 2019.
- [7] 于永辉.基于微服务架构的国有资产管理系统设计实现[D].广西大学, 2021.
- [8] 迟聪慧.当下电商直播发展新趋势分析——以广州市电商直播发展为例[J].新媒体研究.2020(22).
- [9] 高张宝.基于 B/S 架构的企业内部管理系统的设计与实现[D].昆明理工大学, 2019.
- [10]From Digital Divide to Social Inclusion: A Tale of Mobile Platform Empowerment in Rural Areas [J]. Lisha Ye,Huiqin Yang. Sustainability.2020(6).
- [11]高志平.基于 SpringBoot 框架的运维管理系统的设计与实现[D].华东师范大学, 2021.
- [12]崔兰超.B/S 结构下的网络考试系统的实现与应用[J].电脑编程技巧与维护, 2019, No.409(07):35-36+52.
- [13]马雪山, 张辉军, 陈辉等.前后端分离的 Web 平台技术研究与实现[J].电子技术与软件工程, 2022.
- [14]王景.基于 MySQL 的数据库查询性能优化技术研究[J].电脑与电信, 2022.
- [15]周旺, 吴昌平, 李贤和.师生双选系统后端设计[J].无线互联科技, 2021, 18(12): 45-46.